Trabalho Prático: Divisão e Conquista

Projeto e Análise de Algoritmos – Relatório
Ana Clara Macedo (trabalho individual)

1. Introdução

Neste trabalho implementei, em Python, dois problemas resolvidos com a técnica de divisão e conquista (subarranjo de soma máxima e par de pontos mais próximos) e comparei cada um com a solução de força bruta. Para a Etapa 1 também fiz a versão O(n) (algoritmo de Kadane), que é a questão desafio. Depois medi os tempos para entradas de tamanhos crescentes e comparei com o que as recorrências preveem.

Mesmo os problemas não sendo de ordenação, usei como base os códigos de *merge sort* e *quick sort* feitos em aula. A estrutura do *merge sort* (dividir no meio, resolver as duas metades e combinar) é o molde das duas soluções. A função `combinacao` dele é usada na Etapa 2 para intercalar os pontos por y, e o *quick sort* faz a ordenação inicial dos pontos por x. Antes de reaproveitar, corrigi um erro do meu *merge sort*: faltava o `i += 1` na função de combinação, o que causava um laço infinito.

2. Etapa 1: Subarranjo de Soma Máxima

Dado um vetor com as variações diárias de uma ação, queremos o intervalo contínuo [i, j] com a maior soma. A entrada é n seguido dos n inteiros e a saída é `soma inicio fim`, com índices a partir de 0. Como o intervalo não pode ser vazio, se todos os valores forem negativos a resposta é o maior deles sozinho.

Regra de desempate. Quando dois intervalos têm a mesma soma, escolhi devolver o de **menor início** e, se o início também empatar, o de **menor fim**. As três versões seguem essa mesma regra, então elas devolvem exatamente a mesma resposta (isso é verificado nos testes).

2.1 Força bruta – $O(n^2)$

Dois laços: para cada início i, o laço interno anda com j de i até n−1 acumulando a soma (aproveito a soma de [i, j−1] para calcular [i, j] em O(1)). O número de intervalos testados é n + (n−1) + … + 1 = n(n+1)/2, então $T(n) = \Theta(n^2)$. Atualizando só quando a soma é *estritamente* maior, o primeiro máximo encontrado já obedece à regra de desempate.

2.2 Divisão e conquista – O(n log n)

É o mesmo esqueleto do *merge sort*: divido o vetor em [low, meio] e [meio+1, high], resolvo cada metade recursivamente e, no lugar da intercalação, calculo o melhor subarranjo que **cruza o meio**. Para isso ando do meio para a esquerda guardando a maior soma de [i, meio], e do meio+1 para a direita guardando a maior soma de [meio+1, j]. O intervalo que cruza é a junção dos dois, e isso custa O(n). A resposta é a melhor das três (esquerda, cruzando, direita). Para manter o desempate, no lado esquerdo uso >= (prefiro o início mais à esquerda) e no lado direito uso > (prefiro o fim mais à esquerda).

Recorrência: $T(1) = \Theta(1)$ e $T(n) = 2T(n/2) + \Theta(n)$. Pelo Teorema Mestre, a = 2, b = 2 e $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$, que é o caso 2, então **$T(n) = \Theta(n \log n)$**. Pela árvore de recursão dá para ver a mesma coisa: são $\log_2 n$ níveis e, em cada nível, os pedaços somam n elementos, ou seja, cn de trabalho por nível.

2.3 Questão desafio: Kadane – O(n)

Mantenho a melhor soma de um subarranjo que *termina* na posição j. Na posição j+1 só há duas opções: estender esse subarranjo ou começar um novo em j+1. Estender só vale a pena se a soma atual for ≥ 0 (com o "=" o início fica mais à esquerda, que é a regra de desempate). Cada elemento é visto uma vez, então T(n) = T(n−1) + Θ(1) = **Θ(n)**. Um erro comum é começar a melhor soma em 0, o que dá resposta errada quando todos os valores são negativos. Por isso a variável começa em vetor[0].

3. Etapa 2: Par de Pontos Mais Próximos

A entrada é n seguido de n linhas `x y`, e a saída é `distancia i j`, com os índices das duas estações na ordem da entrada. Cada ponto é guardado como a tupla (x, y, índice), o que também resolve empates na ordenação, porque não existem duas tuplas iguais.

3.1 Força bruta – $O(n^2)$

Compara todos os n(n−1)/2 pares e guarda a menor distância, então $T(n) = \Theta(n^2)$.

3.2 Divisão e conquista – O(n log n)

- **Pré-processamento:** ordeno os pontos por x uma única vez com o *quick sort* da aula. Mudei só o pivô, que agora é sorteado: com o pivô fixo no último elemento, uma entrada já ordenada cairia no pior caso $O(n^2)$ e estouraria a pilha de recursão do Python. Com pivô aleatório o custo esperado é O(n log n).
- **Dividir:** a linha vertical passa pelo ponto da mediana (posição meio do vetor ordenado por x).
- **Conquistar:** resolvo cada lado e obtenho $\delta = \min(\delta_{esq}, \delta_{dir})$. Se o subproblema tem n ≤ LIMIAR pontos, uso a força bruta.
- **Combinar:** cada chamada devolve também os seus pontos **ordenados por y**, e junto as duas listas com a `combinacao` do *merge sort* (adaptada para comparar o y) em O(n). Depois filtro a faixa $|x - x_{meio}| < \delta$, que já sai ordenada por y, e comparo cada ponto da faixa com no máximo os **7 seguintes**, parando antes se a diferença em y já for ≥ δ.

Usar a intercalação em vez de reordenar a faixa por y em cada chamada é o que deixa o algoritmo em O(n log n). Se eu ordenasse a faixa toda vez, a combinação custaria O(n log n) e o total seria $O(n \log^2 n)$.

Recorrência: T(n) = Θ(1) para n ≤ LIMIAR (o limiar é uma constante, então a força bruta na folha custa no máximo $LIMIAR^2/2$ comparações) e T(n) = 2T(n/2) + Θ(n), em que o Θ(n) é a intercalação por y, o filtro da faixa e as no máximo 7·|faixa| comparações. Pelo caso 2 do Teorema Mestre, T(n) = Θ(n log n). Somando a ordenação inicial por x, que custa O(n log n) esperado, o total continua **Θ(n log n)**.

3.3 Por que bastam 7 comparações por ponto na faixa

Seja p um ponto da faixa e considere os pontos q com $y_p \le y_q < y_p + \delta$ (os únicos que ainda podem estar a menos de δ de p e vêm depois dele na ordem por y). Todos eles ficam dentro de um retângulo de largura 2δ (δ de cada lado da linha) e altura δ. Divido esse retângulo em **8 quadrados de lado δ/2**, 4 do lado esquerdo e 4 do direito. A diagonal de cada quadrado mede δ/√2 < δ, então dois pontos do mesmo lado dentro do mesmo quadrado estariam a menos de δ um do outro. Isso é impossível, porque δ já é a menor distância dentro de cada lado. Logo, cada quadrado tem no máximo um ponto, o retângulo tem no máximo 8 pontos contando o próprio p, e sobram **no máximo 7 outros**. Por isso basta comparar p com os 7 pontos seguintes da faixa ordenada por y. No código o laço vai até i+7 e ainda para antes quando $y_q - y_p \ge \delta$.

4. Testes

Os casos de teste ficam em `testes/casos_etapa1.txt` (17 casos) e `testes/casos_etapa2.txt` (11 casos), cada um com entrada, saída esperada e uma linha explicando por que o caso existe. O arquivo `test_trabalho.py` roda cada caso chamando o programa pela linha de comando, para testar também a leitura da entrada e o formato da saída, com todas as versões. Os principais casos de borda e a justificativa de cada um:

Caso	Por que testar
E1: n = 1 (positivo e negativo)	caso base da recursão; com valor negativo a resposta não pode virar 0
E1: só negativos (com e sem empate)	resposta é o maior elemento sozinho; pega o erro de começar a soma em 0
E1: empates (intervalos separados, mesmo início, zeros)	verifica a regra de desempate igual nas 3 versões
E1: máximo cruzando o meio / nas pontas	testa a função de cruzamento e os limites do vetor
E1: valores grandes	soma passa de 32 bits (em C/Java daria overflow)
E2: n = 2 e n = 3	menor entrada possível; caso base direto
E2: pontos repetidos	distância 0, faixa vazia ($\delta = 0$) não pode quebrar
E2: par cruzando a linha	só a etapa da faixa encontra a resposta
E2: todos com o mesmo x / mesmo y	a faixa contém todos os pontos; testa o limite de 7 vizinhos
E2: grade 3×3, negativos e decimais	muitos empates; leitura de valores não inteiros
E2: entrada já ordenada por x	seria o pior caso do quick sort sem pivô aleatório

Além disso, há testes aleatórios. A divisão e conquista e o Kadane são comparados com a força bruta em 3.000 vetores pequenos com valores entre -5 e 5 (para forçar muitos empates), e a Etapa 2 em 1.500 conjuntos de pontos com coordenadas em faixas pequenas (muitos pontos repetidos e alinhados) e com limiares de 2 a 64. Também há entradas grandes (até 20.000 elementos, 3.000 pontos já ordenados ou todos no mesmo x). Todos os testes passam.

5. Análise Experimental

Máquina: Sistema: Linux 6.18.44-fc-v185; Processador: Intel(R) Xeon(R) Processor @ 2.10GHz; Nucleos logicos: 2; Memoria: 7.8 GB; Python: 3.13.18 (CPython). **Metodologia:** para cada $n \in \{100, 200, \ldots, 6400\}$ fiz 5 execuções. Cada execução usa uma entrada aleatória nova (inteiros em $[-1000, 1000]$ ou pontos com coordenadas reais em $[0, 10^6]$), e a mesma entrada é dada a todos os algoritmos. Medi com `time.perf_counter()` e com o coletor de lixo desligado. As medições brutas estão em `resultados/medicoes.csv`. Rodei os experimentos duas vezes. A primeira rodada teve mais oscilação (a máquina é compartilhada) e ficou guardada em `medicoes_rodada1_descartada.csv`; os números abaixo são da segunda.

n	E1 bruta	E1 D&C	E1 Kadane	E2 bruta	E2 D&C
100	0,329 ± 0,032	0,163 ± 0,016	0,011 ± 0,001	0,879 ± 0,175	0,325 ± 0,064
200	1,25 ± 0,025	0,317 ± 0,014	0,018 ± 0,001	2,87 ± 0,060	0,519 ± 0,021
400	5,71 ± 0,075	0,724 ± 0,019	0,037 ± 0,002	11,7 ± 0,580	1,24 ± 0,047
800	24,2 ± 0,171	1,55 ± 0,036	0,083 ± 0,021	60,5 ± 19,2	3,53 ± 1,90
1600	98,1 ± 0,127	3,33 ± 0,009	0,154 ± 0,011	217 ± 44,3	6,45 ± 1,72
3200	330 ± 66,1	5,66 ± 1,54	0,255 ± 0,053	835 ± 102	14,5 ± 3,73
6400	1.129 ± 195	10,8 ± 3,35	0,456 ± 0,127	3.533 ± 296	32,8 ± 7,45

Tabela 1 – Tempo médio ± desvio padrão (ms), 5 execuções por tamanho.

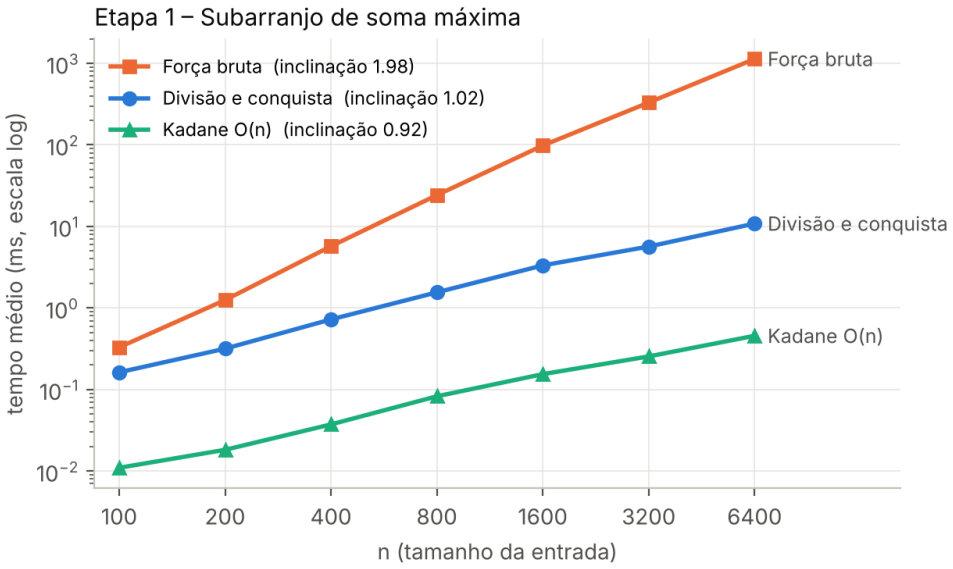


Figura 1 – Etapa 1, escala log-log. A inclinação de cada reta é o expoente de n estimado por mínimos quadrados.

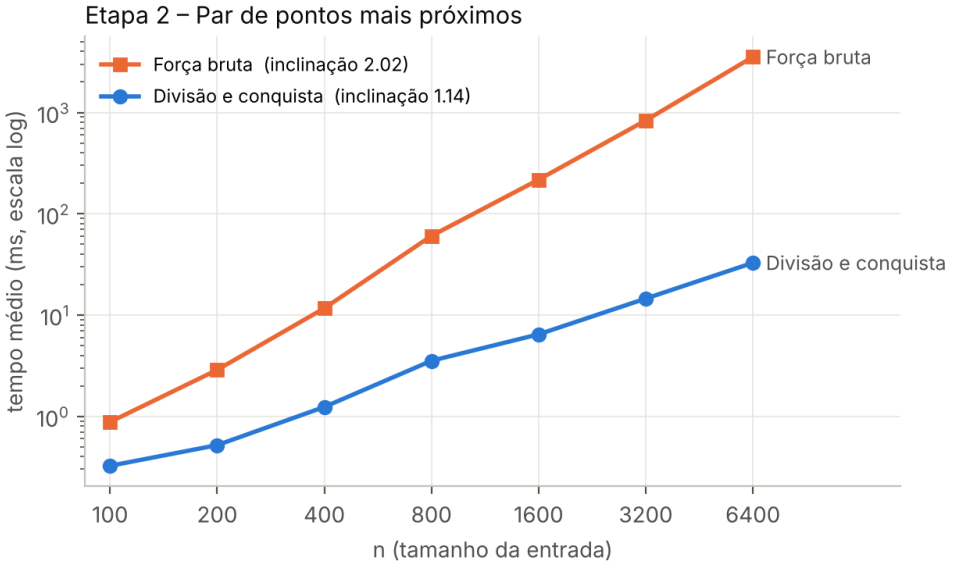


Figura 2 – Etapa 2, escala log-log.

5.1 Comparação com o previsto

Num gráfico log-log, um algoritmo $\Theta(n^k)$ vira uma reta de inclinação k. As forças brutas deram inclinação de aproximadamente 2 nas duas etapas, como previsto. Quando n dobra, o tempo multiplicou em média por 3,91 (Etapa 1) e 4,03 (Etapa 2), e o esperado era 4. Para n log n, dobrar n multiplica o tempo por $2 \cdot \log(2n)/\log n$, que fica entre 2,3 e 2,2 nessa faixa de tamanhos (inclinação esperada ≈ 1,15). As divisões e conquistas multiplicaram em média por 2,02 (Etapa 1) e 2,20 (Etapa 2), e o Kadane por 1,87 (esperado 2).

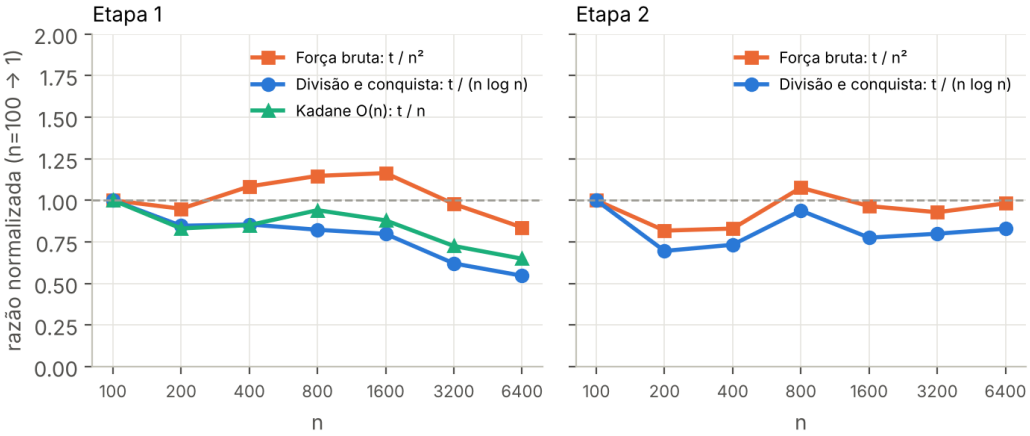


Figura 3 – Tempo dividido pela complexidade prevista, normalizado para 1 em n = 100. Se a análise estiver certa, a curva fica aproximadamente horizontal.

A Figura 3 mostra a mesma coisa de outro jeito. Se o tempo é c·f(n), então t/f(n) deve ficar constante. Para as forças brutas e para a divisão e conquista da Etapa 2 a razão fica perto de 1, ou seja, n^2 e n log n explicam bem os dados. Já a divisão e conquista da Etapa 1 e o Kadane ficaram abaixo do previsto (a razão cai até ≈ 0,55–0,65). Minha interpretação é que, para n pequeno, existe um custo fixo que não depende de n (chamar a função, criar variáveis), e ele pesa mais quando o tempo total é de microssegundos. Esse custo fixo infla o tempo em n = 100, que é o ponto usado para normalizar, e faz a curva parecer descer. No caso da Etapa 1, o fator log n também é pequeno perto do custo das ~2n chamadas recursivas, que é linear, então a curva se parece mais com n do que com n log n nessa faixa. Algumas oscilações isoladas (por exemplo o pulo em n = 800 na Etapa 2) aparecem também no desvio padrão da Tabela 1 e são ruído da máquina, que é virtual e compartilhada.

5.2 Quando a força bruta vence e quando a técnica vence

Para n grande a divisão e conquista ganha com folga: em n = 6400 ela foi cerca de 104 vezes mais rápida na Etapa 1 e 108 vezes na Etapa 2, e a diferença cresce com n porque $n^2/(n \log n) = n/\log n$. Para n pequeno é o contrário (Figura 4, média de 300 execuções por tamanho). A força bruta tem laços simples, sem recursão, sem criar listas e, na Etapa 2, sem ordenar, então o custo por operação dela é bem menor. A divisão e conquista faz menos operações, mas cada uma é mais cara (chamadas de função, fatiamento e intercalação de listas, ordenação inicial). O ponto de cruzamento ficou entre 32 e 64 na Etapa 1 e por volta de 32 na Etapa 2. O Kadane vence as duas em todos os tamanhos, porque faz uma passada só com um laço simples.

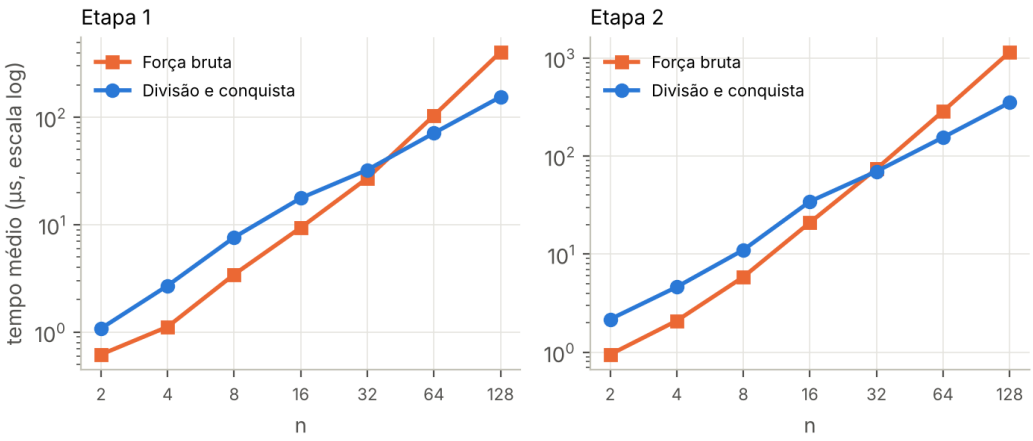


Figura 4 – Tamanhos pequenos (2 a 128): região onde a força bruta é mais rápida.

5.3 Escolha do limiar da Etapa 2

Por causa do que foi visto na Figura 4, vale a pena parar a recursão antes de chegar em 2 ou 3 pontos e resolver o pedaço pequeno por força bruta. Testei limiares de 2 a 128 com n = 1600 e n = 6400. Fiz 20 execuções por limiar (mais que o mínimo, porque as diferenças são pequenas) e intercalei as execuções, rodando a repetição 1 de todos os limiares, depois a repetição 2, e assim por diante, para que uma oscilação da máquina não prejudicasse só um deles. O resultado está na Figura 5.

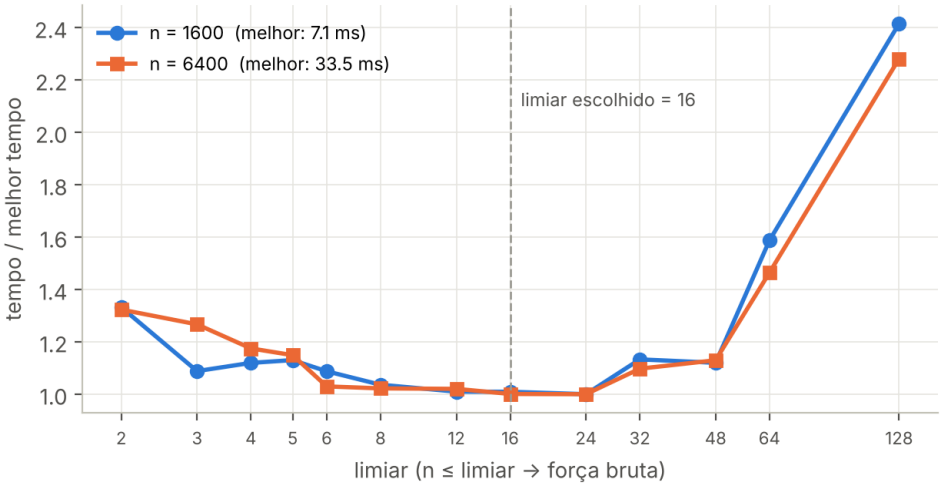


Figura 5 – Tempo de cada limiar dividido pelo melhor tempo do mesmo n.

Limiares muito pequenos são piores: com limiar 2 o tempo foi 1,32× o melhor, por causa de muitas chamadas recursivas para pedaços minúsculos. Limiares grandes também são piores, porque a folha volta a ser quadrática: com limiar 128 o tempo foi 2,28× o melhor. Entre 8 e 24 a curva fica praticamente plana. Escolhi **LIMIAR = 16**, que está no meio dessa região plana e foi o melhor ou empatado com o melhor nos dois tamanhos. Um detalhe que percebi: como $6400/2^9 = 1600/2^7 = 12{,}5$, com qualquer limiar entre 13 e 24 as folhas da recursão têm 12 ou 13 pontos, então a execução é a mesma e os tempos ficam iguais. Pelo mesmo motivo, 32 e 48 dão folhas de 25 pontos e tempos parecidos. Também reparei que o limiar ótimo (≈ 16) é menor que o ponto de cruzamento da Figura 4 (≈ 32). Isso acontece porque, na Figura 4, a divisão e conquista ainda paga a ordenação inicial e a criação das tuplas, que não existem dentro da recursão.

6. Conclusão

As implementações por divisão e conquista ficaram corretas (conferidas com a força bruta em milhares de entradas) e os tempos medidos seguiram as recorrências: inclinação ≈ 2 para as forças brutas e perto de 1 para as versões n log n e para o Kadane. A técnica só compensa a partir de algumas dezenas de elementos, e por isso usar força bruta nos subproblemas pequenos (o limiar) deixou a Etapa 2 cerca de 25% mais rápida que recursar até 2 pontos. Também deu para ver na prática que o *merge sort* e o *quick sort* da aula servem para mais coisas além de ordenar: a ideia de devolver a lista já ordenada e intercalar na volta da recursão foi o que permitiu o O(n log n) do par mais próximo.